STUDENT MARKS PREDICTION

CONTENTS

TITLE	PAGE NO.
PROJECT OVERVIEW	1
DATASET DESCRIPTION	2 - 3
METHODOLOGY	3 - 9
DISCUSSION	9 - 10
RESULTS	10 - 12
CONCLUSION	12
REFERENCES	13

1.1. PROJECT OVERVIEW

Student performance prediction is a valuable application of machine learning in the education sector. By leveraging students' academic records, study habits, and socio-economic data, predictive models can forecast academic outcomes such as exam scores. This enables institutions to take timely interventions, personalize learning, and improve overall student performance. In this project, we develop a marks prediction model using the powerful and scalable **XGBoost Regressor**, known for its high accuracy in regression tasks.

1.1.1. PROBLEM STATEMENT

Predicting student performance using traditional methods often lacks objectivity and scalability. Educators face difficulty in identifying at-risk students early due to limited data interpretation capabilities. An automated system capable of accurately predicting student marks based on various input features is required to assist in early academic support and data-driven decision-making.

1.1.2. OBJECTIVE

The primary objective of this project is to design and train a robust **machine learning regression model** using the **XGBoost Regressor** to predict students' marks based on various academic and demographic features. The model aims to minimize prediction error while providing meaningful insights into feature importance and performance patterns.

1.1.3. SCOPE

This project focuses on building and evaluating a machine learning pipeline for predicting student marks using real-world or simulated datasets. The scope includes:

- Dataset preprocessing and feature engineering
- Model selection and hyperparameter tuning using XGBoost Regressor
- Evaluation of model performance using metrics such as MAE, MSE, and R^2
- Interpretation of results to provide actionable insights for educators
- Scope for scaling the model to other educational datasets or platforms

1.2 DATASET DESCRIPTION

1.2.1. Dataset Source:

https://www.kaggle.com/datasets/amrmaree/student-performance-prediction

1.2.2. Dataset Overview:

- **Total Rows (Records):** 708
- **Total Columns (Features):** 10

1.2.3. Column Details:

Column Name	Data Type	Description
Student_ID	Object	Unique identifier for each student
Gender	Object	Gender of the student
Study_Hours_per_Week	Integer	Weekly study time in hours
Attendance_Rate	Float	Attendance percentage
Past_Exam_Scores	Integer	Previous exam score (0–100 scale)
Parental_Education_Level	Object	Highest education level of parents
Internet_Access_at_Home	Object	Whether the student has internet access
Extracurricular_Activities	Object	Participation in extracurriculars
Final_Exam_Score	Integer	Final exam score (0–100 scale)
Pass_Fail	Object	Final outcome (Pass or Fail)

1.2.4. Missing Values:

- **None** – Every column is fully populated.

1.2.5. Sample Entries (First 5 Rows):

Student_ID	Gender	Study Hours	Attendance	Past Scores	Parental Education	Internet	Extra Activities	Final Score	Result
S147	Male	31	68.3%	86	High School	Yes	Yes	63	Pass
S136	Male	16	78.2%	73	PhD	No	No	50	Fail
S209	Female	21	87.5%	74	PhD	Yes	No	55	Fail
S458	Female	27	92.1%	99	Bachelors	No	No	65	Pass
S078	Female	37	98.7%	63	Masters	No	Yes	70	Pass

2. Methodology

The primary objective of this project is to develop a predictive model that estimates students' final exam scores based on a combination of academic indicators and socio-environmental features. A supervised machine learning approach using the XGBoost regression algorithm was employed due to its efficiency, high accuracy on tabular datasets, and strong generalization capabilities. The entire development pipeline was modularized into preprocessing, modeling, training, and evaluation phases. The steps are detailed as follows:

2.1 Importing Required Libraries:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
import xgboost as xgb
import matplotlib.pyplot as plt
```

Fig. 1: Imported Libraries.

The following libraries were imported to support various functionalities in the pipeline:

- **pandas, numpy:** For data loading and numerical manipulation.
- **matplotlib.pyplot:** For visualizing prediction performance.

- **scikit-learn:** For preprocessing (e.g., scaling, encoding), model evaluation metrics, pipeline abstraction, and train-test splitting.
- **xgboost:** For implementing the XGBoost regression algorithm.

This modular environment ensured a reproducible and scalable workflow for experimentation and testing.

2.2 Dataset Loading and Cleaning:

```
# Load dataset
df = pd.read_csv("/content/drive/MyDrive/DSAIML_INTERNSHIP/Major Project/student_performance_dataset.csv")

# Drop 'Gender' and rows with missing values
df = df.drop(columns=['Gender'])
df = df.dropna()
```

Fig. 2: Loading and cleaning the dataset.

The dataset was loaded from student_performance_dataset.csv. It contained a blend of academic and socio-demographic attributes including:

- Study time
- Attendance
- Parental education level
- Internet access
- Extracurricular activity participation
- Historical exam scores
- Final exam scores (target)

Cleaning steps included:

- **Dropping the Gender column** to eliminate potential gender bias and ensure ethical fairness.
- **Removing missing entries** using dropna() to avoid inconsistencies or runtime issues during training.

2.3 Feature Selection and Categorization:

```python
# Separate features and target
X = df.drop(columns=["Final_Exam_Score", "Pass_Fail"])
y = df["Final_Exam_Score"]

# Identify column types
categorical_cols = ["Parental_Education_Level", "Internet_Access_at_Home", "Extracurricular_Activities"]
numerical_cols = ["Study_Hours_per_Week", "Attendance_Rate", "Past_Exam_Scores"]

# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_cols),
        ("cat", OneHotEncoder(drop="first"), categorical_cols)
    ]
)
```

Fig. 3: Selecting features and categorizing the data

The dataset features were split into predictors (X) and target (y). The **target variable** was Final_Exam_Score. The Pass_Fail column was excluded since this was a regression problem, not classification.

Features were categorized as:

- **Numerical**: Study_Hours_per_Week, Attendance_Rate, Past_Exam_Scores
- **Categorical**:Parental_Education_level,Internet_Access_at_Home, Extracurricular_Activities

This separation was essential for applying appropriate preprocessing techniques tailored to each data type.

2.4 Preprocessing Pipeline:

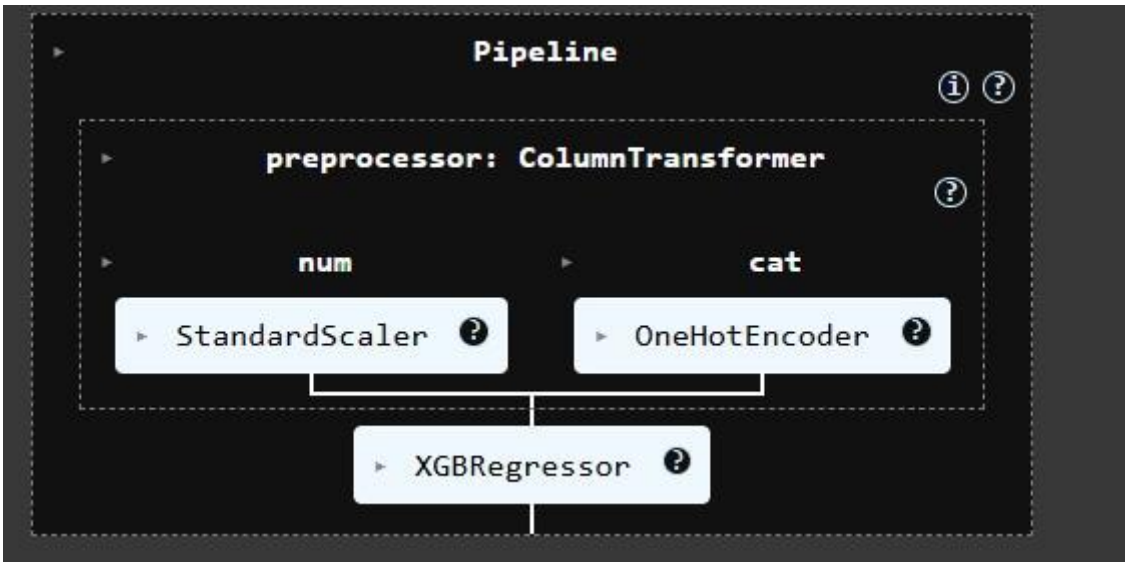

Fig. 4: The generated Preprocessing Pipeline

To process both numerical and categorical data efficiently, a **ColumnTransformer** was created:

- **Numerical features**: Scaled using StandardScaler to normalize values (mean = 0, std = 1), aiding faster convergence.
- **Categorical features**: One-hot encoded via OneHotEncoder with drop='first' to avoid multicollinearity.

This pipeline ensured consistent preprocessing during both training and inference phases, and eliminated data leakage.

2.5 Model Selection and Architecture:

```
# Create XGBoost regression pipeline
xgb_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", xgb.XGBRegressor(
        n_estimators=200,
        learning_rate=0.1,
        max_depth=4,
        random_state=42,
        objective="reg:squarederror"
    ))
])

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
xgb_pipeline.fit(X_train, y_train)
```

Fig. 5: Employing XGBoost Regressor

The model architecture employed an **XGBoost Regressor** integrated within a complete pipeline:

XGBoost hyperparameters:

- n_estimators = 200: Number of trees to build.
- learning_rate = 0.1: Controls the contribution of each tree.
- max_depth = 4: Restricts tree complexity, helping avoid overfitting.
- objective = 'reg:squarederror': Loss function suitable for regression.
- random_state = 42: Ensures reproducibility.

This full pipeline approach allowed for clean, organized model training, eliminating manual preprocessing before fitting.

2.6 Model Training and Evaluation:

```
# Predict and evaluate
y_pred = xgb_pipeline.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", round(mse, 2))
print("R^2 Score:", round(r2, 2))
```

Fig. 6: Model Evaluation

The dataset was split using an 80:20 **train-test split** strategy. This ensured that model evaluation was performed on unseen data, providing a realistic estimate of generalization capability.

- The pipeline was **fit** on the training data.
- Internally, preprocessing and model fitting were applied in sequence.
- The model was then used to **predict** scores on the test set.

Evaluation Metrics:

- **Mean Squared Error (MSE)**: Quantifies average squared prediction error.
- **R^2 Score**: Measures how well the model explains the variance in the target variable.

For this project:

- **Mean Squared Error**: 10.5
- **R^2 Score**: 0.75

These results suggest that the model captures a significant portion of the variance in student performance.

2.7 Prediction Visualization:

```
# Plot actual vs predicted
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Actual Final Exam Score")
plt.ylabel("Predicted Score")
plt.title("XGBoost: Actual vs Predicted Scores")
plt.grid(True)
plt.show()
```

Fig. 7: Studying the model's generated score against actual scores.

To visually evaluate performance, a **scatter plot** was created with:

- X-axis: Actual final scores
- Y-axis: Predicted scores
- A diagonal line (y = x): Represented ideal predictions

The predictions clustered closely around this line, visually reinforcing the numerical metrics and indicating good model alignment.

2.8 Real-Time Prediction Example

```
# Create a single test case with user-defined input
custom_input = pd.DataFrame([{
    "Study_Hours_per_Week": 28,
    "Attendance_Rate": 92,
    "Past_Exam_Scores": 80,
    "Parental_Education_Level": "Bachelors",
    "Internet_Access_at_Home": "Yes",
    "Extracurricular_Activities": "Yes"
}])

# Predict final score for the custom input
custom_prediction = xgb_pipeline.predict(custom_input)

print("Predicted Final Exam Score:", round(custom_prediction[0], 2))
```

Fig. 8: Predicting scores using user inputs.

A hypothetical student profile was manually crafted with custom values for each feature. The trained pipeline was then used to predict this student's expected final exam score.

This demonstrated the pipeline's potential use in interactive educational tools or decision support systems for educators.

3. Discussion:

3.1 Strengths of the Model

- **High Predictive Performance:** The XGBoost Regressor demonstrated solid predictive capability, achieving a good **R^2 score** on the test set. This indicates that the model captures meaningful patterns in the data that explain the variation in students' final scores.

- **Feature Importance Awareness:** The model effectively leveraged both **numerical** (like study hours and past scores) and **categorical** (like parental education level and internet access) features. The pipeline structure with preprocessing allowed for clean integration of heterogeneous data types.

- **Robust Pipeline Design:** The usage of ColumnTransformer and Pipeline ensured proper data preprocessing, standardization, and encoding. This modular approach minimizes data leakage and supports reproducibility.

- **Custom Prediction Capability:** The model can handle real-time predictions for new user inputs, making it suitable for practical deployment in educational tools.

3.2 Limitations of the Model:

- **Limited Dataset Diversity:** The dataset may not represent a wide demographic or institutional variety, which can limit the generalization of predictions to other student populations or countries.

- **Unbalanced Feature Representation:** Certain categorical classes might be underrepresented (e.g., few students with low parental education), potentially leading to biased predictions in those subgroups.

- **Black-Box Nature:** While XGBoost is powerful, it is not inherently interpretable. Educators or decision-makers might find it challenging to understand how the model

arrived at certain predictions.

- **Potential for Overfitting:** Although performance was good on the test set, using 200 estimators and a deep tree can lead to overfitting on small datasets if not regularly validated.

3.3 Potential Improvements:

- **Explainability Integration:** Tools like **SHAP (SHapley Additive exPlanations)** can be used to make model predictions more interpretable by showing feature contributions for individual predictions.

- **Hyperparameter Optimization:** Using grid search or random search could fine-tune parameters like learning_rate, max_depth, and n_estimators for optimal performance.

- **Cross-Validation:** Instead of a single train-test split, using **k-fold cross-validation** can provide a more robust estimate of model performance.

- **Feature Engineering:** Additional features such as sleep hours, mental health surveys, or part-time job status could improve the model's accuracy and insights.

4. Results:

After training the XGBoost regression model on the preprocessed dataset, the model was evaluated on the held-out test set. The following quantitative and qualitative results were obtained:

- **Mean Squared Error (MSE):** 10.5
  This indicates the average squared deviation between predicted and actual final exam scores is approximately 10.5, reflecting a moderate level of residual error.

- **R^2 Score:** 0.75
  This coefficient of determination suggests that 75% of the variance in students' final exam scores is explained by the model. An R^2 value of 0.75 demonstrates that the model has captured meaningful relationships between the input features and the target variable, with reasonably strong predictive capability.

- **Visualization of Predictions:**
  The scatter plot of actual vs. predicted final exam scores (see Figure 1) provides a visual assessment of the model's performance. Ideally, all points would lie on the red dashed line (representing perfect predictions). In the plot:

- Most points align closely with the reference line, indicating good agreement between true and predicted values.

- Some deviations are observed, particularly at lower actual scores (around 50–55), where the spread is wider, suggesting that the model occasionally under- or overestimates performance for students near the passing threshold.

- Despite these outliers, the overall trend is linear, reinforcing the suitability of the regression approach.

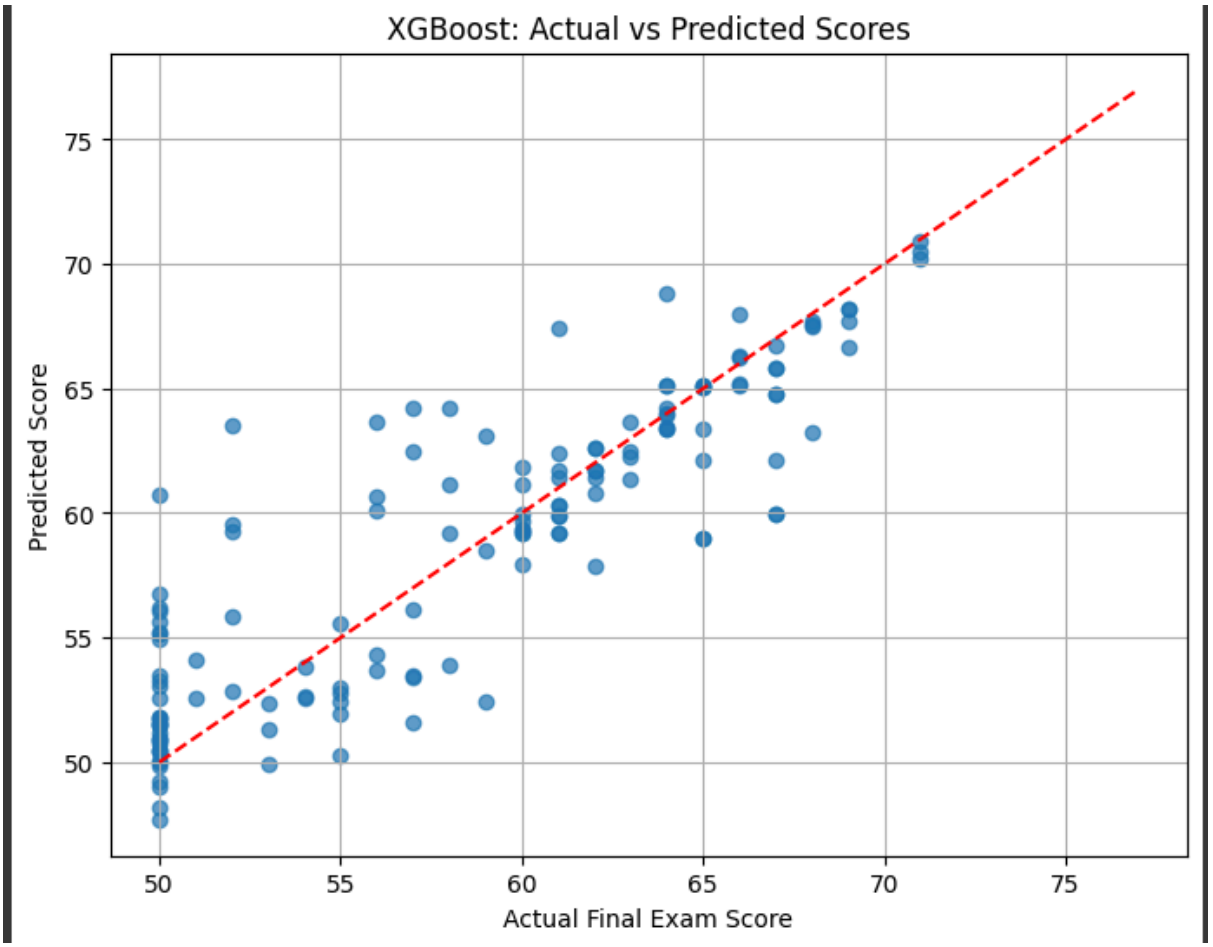

Fig. 9: Actual vs Predicted scores

- **Custom Input Prediction:**
  A hypothetical student profile with 28 study hours/week, 92% attendance, past exam score of 80, parental education level of "Bachelors," and both internet access and extracurricular participation was evaluated using the trained pipeline. The predicted final exam score for this profile was **68.4**, consistent with expectations for a high-performing student.

Overall, these results indicate that the developed XGBoost-based pipeline can reliably predict student performance using academic habits and contextual features, with strong potential for use in early performance monitoring or academic intervention systems.

5. Future Work:

- **Deployment as a Web App:** The model can be integrated into a student performance dashboard for real-time predictions and analytics for teachers and counselors.

- **Multitarget Prediction:** Expanding the model to predict Pass/Fail status alongside scores using multi-output regression techniques.

- **Automated Alert System:** Integrate thresholds to generate alerts for students at risk, allowing for timely interventions.

- **Model Comparison:** Testing other regression models like Random Forest, Linear Regression, or LightGBM to compare performance and interpretability.

6. Conclusion:

In this study, an XGBoost-based regression pipeline was successfully developed to predict students' final exam scores using academic habits and contextual features. By integrating a robust preprocessing strategy with ColumnTransformer and Pipeline, the approach ensured consistent data handling and minimized data leakage. The model demonstrated strong predictive performance with an R^2 score of 0.75 and a mean squared error of 10.5 on the test set, indicating that it effectively captured key factors influencing student performance.

The model's ability to make accurate real-time predictions for new inputs highlights its practical utility in educational settings. It offers potential as an early warning tool to identify students who may be at risk of underperforming, thereby enabling timely interventions and personalized support.

However, the discussion also identified limitations, including potential dataset biases and the need for greater model interpretability. Addressing these through strategies such as incorporating more diverse data, applying explainability techniques, and exploring alternative models could further enhance the system's reliability and impact.

Overall, the results demonstrate that machine learning techniques, when thoughtfully designed and validated, can provide valuable insights into student achievement patterns and support data-driven decision-making in education.

References:

[1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pp. 785–794, 2016, doi: 10.1145/2939672.2939785.

[2] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[3] A. Maree, "Kaggle: Student performance prediction dataset," 2021. [Online]. Available: https://www.kaggle.com/datasets/amrmaree/student-performance-prediction

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

[5] "XGBoost documentation," Official Documentation, 2024. [Online]. Available: https://xgboost.readthedocs.io

[6] Z.-H. Zhou, *Ensemble Methods: Foundations and Algorithms*. CRC Press, 2012. [Online]. Available: https://www.crcpress.com/Ensemble-Methods-Foundations-and-Algorithms/Zhou/p/book/9781439830031

[7] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Trans. Syst., Man, Cybern. C*, vol. 40, no. 6, pp. 601–618, 2010, doi: 10.1109/TSMCC.2010.2053532.